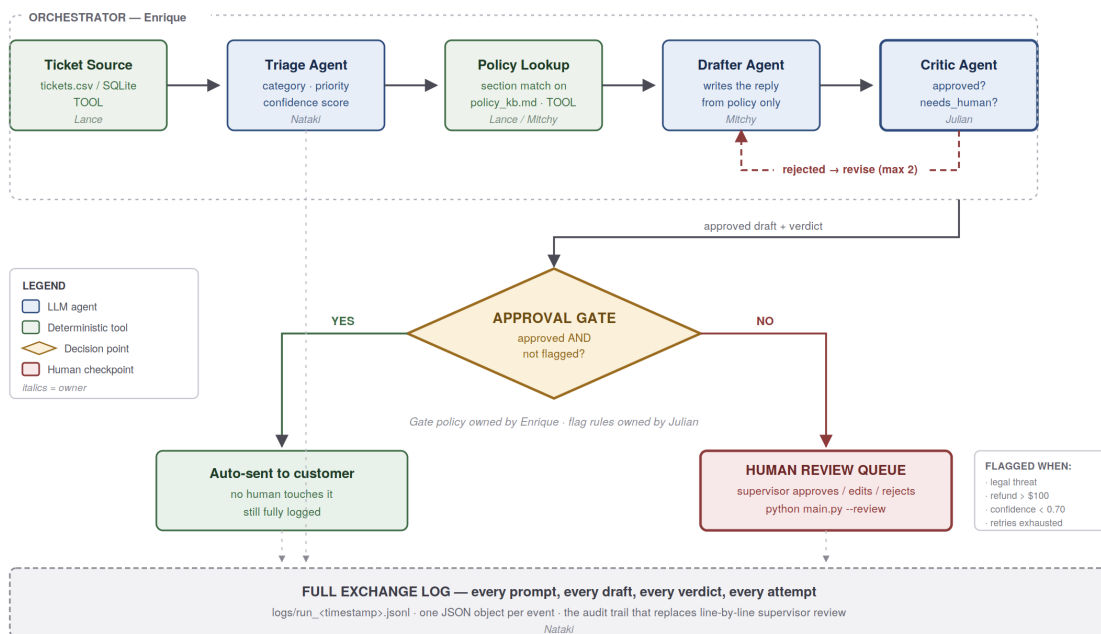


Northstar Support Co.

Agentic Ticket Triage — Team Build Guide

Northstar Support Co. — Agentic Ticket Triage

Multi-agent pipeline with a critic gate. The human checkpoint moves from every reply to flagged replies only.



Enrique Quezada · Lance Gonzalez · Julian Seiferth

Mitchy Derosé · Nataki Boykin

Contents

Start Here	Everyone
Setup From Zero	Everyone — do this first
Glossary	Everyone — look things up
Git Reference	Everyone
The Code Concepts You'll Need	Everyone
Enrique	Orchestrator Engineer & Repo Owner
Lance	Integration Engineer
Mitchy	Prompt Engineer
Julian	QA / Critic Engineer
Nataki	Logging & Observability Engineer

How to use this guide. If you have never programmed before, start with **Setup From Zero** on page 7 — it assumes nothing and walks through installing everything, opening a terminal, and making your first commit. Everyone reads the first five sections. After that, find the section with your name on it and work through the numbered tasks in order. Each personal section ends with a checklist, the one line you own in the pitch, and the specific errors you are likely to hit.

You are not starting from a blank file. Every piece of the project already exists as a working stub. Your job is to replace your stub with the real thing. If you get stuck, the project still runs.

START HERE

Everyone reads this page. Then you read your own guide in `docs/guides/`.

You are	Your guide
Enrique	<code>docs/guides/ENRIQUE.md</code>
Lance	<code>docs/guides/LANCE.md</code>
Mitchy	<code>docs/guides/MITCHY.md</code>
Julian	<code>docs/guides/JULIAN.md</code>
Nataki	<code>docs/guides/NATAKI.md</code>

What we're building, in three sentences

Northstar Support has six people who read every support ticket, tag it, look up the policy, write a reply, and get a supervisor to check it. It works, but a ticket takes 18 hours. We're building agents to do the reading, tagging, lookup, and drafting, keeping a human checkpoint only on the replies that actually need one.

The code already runs. Nobody is starting from a blank file. Every piece exists as a working stub, and your job is to replace your stub with the real thing. If you get stuck, the project still runs, just with your stub in place.

Never programmed before? Start there instead.

If you have never used a terminal, installed Python, or used git, do `docs/SETUP_FROM_ZERO.md` first. It assumes nothing: installing Python and VS Code, what a terminal is, how to move between folders, and a practice commit with nothing at stake. Takes about 60-90 minutes.

Come back here when `python main.py --mock` works on your machine.

There's also `docs/GLOSSARY.md` — every term we use, in plain language. Look things up there rather than guessing from context.

Day 0: everyone does this, before anything else

Takes about 20 minutes if you've done this kind of setup before. If you haven't, use `docs/SETUP_FROM_ZERO.md` instead.

```
# 1. Get the code
git clone https://github.com/<enrique>/northstar-triage.git
cd northstar-triage

# 2. Install exactly the pinned versions
pip install -r requirements.txt

# 3. Prove it works
python main.py --mock
```

You should see 12 tickets scroll past, some marked AUTO and some marked HUMAN, then a summary. **If you see that, you are set up correctly.** If you don't, post the entire error message in the group chat, not just the last line.

Then read, in this order:

1. `README.md` — 5 minutes
2. `docs/GLOSSARY.md` — skim it, then keep it open while you work
3. `docs/CONCEPTS.md` — 20 minutes, the Python and LangChain constructs in this repo you may not have seen before
4. `docs/GIT_GUIDE.md` — 15 minutes, before your first commit
5. Your own guide in `docs/guides/`

Mock mode is your friend

```
python main.py --mock          # canned responses, no API key, free, instant
```

Build and test everything in mock mode first. Only switch to real API calls when your logic already works. Real calls cost money and are slow, and you do not want to be debugging a typo at 30 seconds a run.

The one rule

Never commit to `main`. Work on your own branch, open a pull request, Enrique merges it. Full instructions in `docs/GIT_GUIDE.md`.

Order of work: who's blocked on whom

Nobody is blocked right now. Every stub returns something sensible, so all five of us can work at the same time from day one. That's deliberate.

Two soft dependencies to be aware of:

- **Mitchy and Julian both need real API calls** to tune prompts. Mock responses are canned, so they can't tell you whether a prompt is any good. Enrique gets the shared API key sorted in Week 1.

- **Julian's labeled ticket set** makes everyone's tuning measurable. Once it exists, "did my change help?" has a real answer instead of a vibe. That's why the group labeling hour is early.

Rough two-week shape

Week 1 — make your piece real

- Everyone: Day 0 setup, read your guide
- Enrique: repo live, collaborators added, API key distributed
- Group: one-hour ticket labeling session (see Julian's guide)
- Everyone: first PR merged, however small

Week 2 — tune, measure, write

- Everyone: your piece working against real API calls
- Julian: accuracy numbers for the deck
- Group: written deliverables
- Enrique: integration test, demo rehearsal

Get one small PR merged in the first few days, even a one-line comment fix. The first PR is the scary one. Do it early, when nothing is at stake.

Written deliverables

Deliverable	Owner
Architecture diagram	Enrique (done, docs/architecture.png)
Before/After one-pager	Mitchy
"What We Keep From the Old Process"	Nataki
5-minute stakeholder pitch deck	Julian
Labeled evaluation set	Julian (group hour)

The people with the lightest code load carry more of the writing. That's on purpose.

When you're stuck

1. Run `python main.py --mock`. Does the project still work?
2. Add a `print()` to see what a variable actually contains. Not cheating.
3. Post in the group chat with: what you were trying to do, the command you ran, and the **entire** error message.

4. Nobody on this team has done this before. Asking early is faster than struggling quietly, and it is not a sign of anything.

Setup From Zero

For everyone, before you open your own section.

This assumes you have never written code, never used a terminal, and have never touched git. If that's you, you're in the right place and nothing here is obvious. If you've done some of this before, skim and skip ahead.

Set aside 60–90 minutes. Do it in one sitting if you can. **Getting stuck here is normal and is not a sign of anything** — installing developer tools is genuinely the worst part of programming, and it gets much better right after.

Part 0: What these things even are

Five tools. Here's what each one is for, in plain terms.

Tool	What it actually is
Python	The language our project is written in. Installing it gives your computer the ability to run our code.
VS Code	A text editor built for code. Like Word, but for programs. It also has a terminal built into it, which saves you learning a separate app.
Terminal	A window where you type commands instead of clicking buttons. It's how you tell the computer to run things.
git	Tracks every change to the code and lets five people work on it without overwriting each other.
GitHub	A website that stores the shared copy of our code. git is the tool, GitHub is the place.

A folder is a folder. Our project is a folder on your computer with files in it. The terminal is just a way of moving around folders by typing instead of double-clicking. That's the whole idea.

Part 1: Install Python

On a Mac

Your Mac already has an old Python, but we need a current one.

1. Go to **python.org/downloads**
2. Click the big yellow button (it detects your Mac automatically)
3. Open the downloaded `.pkg` file and click through the installer
4. **Restart your computer.** Skipping this causes a confusing error later.

On Windows

1. Go to **python.org/downloads**
2. Click the big yellow button
3. Run the downloaded `.exe`

4. **On the very first screen, check the box that says "Add python.exe to PATH".** It's at the bottom and easy to miss. If you don't check it, your computer won't be able to find Python and nothing else in this guide will work. If you already installed without it, run the installer again and choose Modify.
5. Then click Install Now
6. **Restart your computer.**

Part 2: Install VS Code

1. Go to **code.visualstudio.com**
2. Download and install it
3. Open it. You'll see a mostly empty window with a sidebar. That's correct.

Install the Python extension (this gives you helpful colors and error underlines):

1. Click the icon in the left sidebar that looks like four squares (Extensions)
2. Type **Python** in the search box
3. Install the first result, the one published by Microsoft

Part 3: Open a terminal and find out where you are

This is the part that feels alien. It gets familiar fast.

In VS Code, press **Ctrl + `** (that's the backtick key, above Tab, left of the 1 key). Or use the menu: **Terminal → New Terminal**.

A panel opens at the bottom with some text and a blinking cursor. That's your terminal. You type a command, press Enter, and it does something.

Try three commands. Type each one, press Enter, look at what comes back.

```
pwd
```

"Print working directory." It shows which folder you're currently standing in. On Windows use **cd** by itself instead of **pwd**.

```
ls
```

"List." It shows the files and folders where you're standing. On Windows, use **dir** instead.

```
cd Desktop
```

"Change directory." It moves you into a folder named Desktop. Now run **pwd** again and you'll see you moved.

```
cd ..
```


Two dots means "go up one folder," back to where you were.

That's the entire terminal skill you need for this project. Where am I (`pwd`), what's here (`ls`), move somewhere (`cd`). Everything else you'll be copy-pasting.

Part 4: Check Python is installed

In the terminal:

Mac:

```
python3 --version
```

Windows:

```
python --version
```

You should see something like `Python 3.12.4`. Any version starting with 3.11, 3.12, or 3.13 is fine.

Important for the rest of this project: on Mac you type `python3` and `pip3`. On Windows you type `python` and `pip`. Our guides are written with `python` and `pip`. **Mac users: add the 3 every time.** This one difference causes more confusion than anything else on this list.

If you get "command not found" or "not recognized," Python either didn't install or you skipped the restart. Restart first, then try again. If it still fails on Windows, you almost certainly missed the "Add to PATH" checkbox.

Part 5: Check git

```
git --version
```

Mac: if git isn't there, a popup appears offering to install developer tools. Click Install and wait. That's the easiest path.

Windows: if it's missing, go to git-scm.com/downloads, install it, and accept every default. Then close VS Code completely and reopen it, or the terminal won't see git yet.

Then tell git who you are. Use your real name and the email on your GitHub account:

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

This is what puts your name on your commits. Do it once, never again.

Part 6: Get the project onto your computer

Decide where it goes first. Your Desktop is fine and easy to find.

```
cd Desktop
```

Now copy the project down from GitHub. Enrique will post the exact URL — use his, not this placeholder:

```
git clone https://github.com/<enrique>/northstar-triage.git
```

You'll see a few lines about counting and unpacking objects. When it finishes, there's a new folder on your Desktop called `northstar-triage`.

Move into it:

```
cd northstar-triage
```

Every command for the rest of this project gets run from inside this folder. If a command fails with "file not found," the first thing to check is whether you're standing in the right place. Run `pwd` and look.

Open the project properly in VS Code: File → Open Folder → pick `northstar-triage`. The sidebar now shows all our files. Click any of them to read it. Reopen the terminal (Ctrl + `) and notice it already starts you inside the project folder. That's why we open the folder rather than individual files.

Part 7: Install what the project needs

Our project uses a few libraries other people wrote. This downloads them:

Mac:

```
pip3 install -r requirements.txt
```

Windows:

```
pip install -r requirements.txt
```

You'll see a wall of scrolling text. That's normal. It ends with something like `Successfully installed langchain-openai-1.5.1 ...`.

If you get a permissions error or a message about an "externally managed environment," add this to the end and run it again:

```
--break-system-packages
```

Part 8: Run it

The moment of truth:

Mac:

```
python3 main.py --mock
```

Windows:

```
python main.py --mock
```

You should see 12 tickets scroll past, each labeled AUTO or HUMAN, then a summary with an automation rate.

If you see that, you are fully set up. Everything from here is editing text files and re-running that command.

`--mock` means "use fake responses instead of calling the real AI." It's free, instant, and needs no key. Use it constantly.

Part 9: Your practice commit (do this before any real work)

You're going to make one tiny change and push it through the entire git workflow. Nothing here can break anything. The point is to have already done it once before it matters.

Step 1 — make your own branch. Replace `yourname`:

```
git checkout -b yourname/practice
```

You should see `Switched to a new branch 'yourname/practice'`. A branch is your own private copy of the project. Nothing you do here affects anyone else until it gets merged.

Step 2 — make a change. In the VS Code sidebar, click `TEAM.md`. Find your name and fill in the blank next to it. One sentence, anything you like.

Step 3 — save. Ctrl+S (Cmd+S on Mac). VS Code does not auto-save. Unsaved files are a classic source of "why isn't my change working."

Step 4 — see what git noticed:

```
git status
```

It should list `TEAM.md` under "modified." git is watching every file and it noticed you touched that one.

Step 5 — stage and commit:

```
git add TEAM.md
git commit -m "Add my line to TEAM.md"
```

`add` means "include this in what I'm about to save." `commit` means "save it, with a note about what I did."

Step 6 — push it to GitHub:

```
git push -u origin yourname/practice
```

The `-u origin yourname/practice` part is only needed the first time you push a new branch. After that, plain `git push` works.

Step 7 — open a pull request. Go to the repo on github.com. A yellow banner appears near the top: "*yourname/practice had recent pushes.*" Click **Compare & pull request**, then **Create pull request**. Then tell the group chat.

That's it. That's the entire workflow you'll use for every piece of real work. You just did the scary part with nothing at stake.

Step 8 — go back to main and get everyone's changes:

```
git checkout main
git pull
```

Do this every single time before starting new work.

Part 10: Your rhythm from now on

Every time you sit down to work:

```
git checkout main
git pull
git checkout -b yourname/what-im-doing
```

...do your work, then:

```
git add .
git commit -m "what I did"
git push -u origin yourname/what-im-doing
```

...then open the pull request on github.com and tell the chat.

Print this. Tape it to your monitor. You'll internalize it in about a week.

Errors you will hit, and what they mean

command not found: python3 / 'python' is not recognized Python isn't installed, or you skipped the restart, or (Windows) you missed the "Add to PATH" checkbox. Restart first.

No such file or directory: main.py You're standing in the wrong folder. Run `pwd` (Mac) or `cd` (Windows). You need to be inside `northstar-triage`.

ModuleNotFoundError: No module named 'pydantic' Part 7 didn't finish. Run the `pip install -r requirements.txt` line again and read the end of the output for an error.

fatal: not a git repository Same problem as above — you're outside the project folder. `cd` into it.

error: pathspec 'main' did not match You typo'd a branch name. Run `git branch` to see the branches that exist.

Permission denied (publickey) or a login prompt on push GitHub needs to know it's you. The simplest fix is installing **GitHub Desktop**, signing in there once, and letting it handle credentials for you. Then your terminal pushes will work.

Your branch is behind 'origin/main' Someone merged something since you last pulled. Run `git pull`.

Nothing happens when I run the file Check you saved (Ctrl+S). VS Code shows a filled dot in the file's tab when there are unsaved changes.

When you're stuck

Post in the group chat with all three of these:

1. What you were trying to do
2. The **exact command** you typed
3. The **entire** error message, copy-pasted, not just the last line

The lines above the final error usually name the file and line number, which is where the actual answer is. A screenshot of the whole terminal is fine too.

Nobody on this team has done this before. Asking after fifteen minutes of being stuck is the correct move, not a last resort.

Glossary

Every term we use in the guides, in plain language. Look things up here instead of guessing from context.

The tools

Terminal (also: command line, shell, console) — a window where you type commands instead of clicking. Open it in VS Code with **Ctrl + `**.

VS Code — the editor we write code in. It has the terminal built in, which is why we use it.

Directory — a folder. Same thing. Programmers say directory.

Path — a folder's address, like `Desktop/northstar-triage/agents/critic.py`.

Repository (repo) — the project folder, with its whole history tracked by git.

Library / package / dependency — code someone else wrote that we use. `pip install` downloads them. LangChain and Pydantic are ours.

`pip` — the tool that installs Python libraries. On a Mac you type `pip3`.

git and GitHub

git — the tool that tracks changes. Runs on your computer.

GitHub — the website that hosts the shared copy. git is the tool, GitHub is the place.

clone — download the project to your computer for the first time.

branch — your own private copy of the project to work in. Nothing you do on your branch affects anyone else until it's merged.

`main` — the official branch. It must always work. Never commit to it directly.

commit — save a snapshot of your changes, with a note describing them.

stage (`git add`) — mark which changes you want in your next commit.

push — send your commits up to GitHub.

pull — download everyone else's commits to your computer.

pull request (PR) — asking for your branch to be merged into `main`. Enrique reviews and merges.

merge — combine your branch's work into `main`.

merge conflict — two people changed the same lines and git can't decide which version wins. You'll see `<<<<<<` markers in the file. It's a question, not a disaster.

Python

Variable — a name for a value. `count = 5`.

Function — a named block of code you can run. `run_triage(ticket)` calls the function `run_triage` and hands it a ticket.

Argument / parameter — the values you hand a function. In `load_tickets(limit=5)`, `limit=5` is an argument.

Return — what a function hands back when it's done.

Import — bring in code from another file. `from state import Ticket`.

Module — a single `.py` file.

String — text. Always in quotes: `"billing"`.

Integer / float — a whole number / a decimal number.

Boolean — `True` or `False`.

List — an ordered collection: `["billing", "technical"]`.

Dictionary (dict) — labeled values: `{"category": "billing"}`.

Class — a blueprint for making objects. Our Pydantic models in `state.py` are classes; a `Ticket` you create from one is an object.

Object / instance — a specific thing made from a class.

Docstring — the `"""triple-quoted text"""` at the top of a file or function explaining what it does.

Exception / error / traceback — what you get when something breaks. The traceback is the list of lines above the error, and it tells you where it happened.

`try` / `except` — "attempt this; if it breaks, do that instead" rather than crashing.

This project

Agent — one AI worker with one job and its own instructions. We have three: triage, drafter, critic.

System prompt — the instructions telling an agent who it is and what to do. Ours live in `prompts/`.

Orchestrator — the code that decides which agent runs when. Ours is `orchestrator.py`. It's regular Python, not an AI.

Shared state — the object passed between agents so each one can see what the last one did. Ours is `TicketRun` in `state.py`.

Handoff — what gets passed from one agent to the next. Deciding what to include, and what to leave out, is a design decision we defend in the pitch.

Critic — the agent that reviews the drafter's work before it ships. Stands in for the human supervisor.

Retry loop — when the critic rejects a draft, the drafter tries again with the critic's notes. Ours allows 2 attempts.

Approval gate — the decision point at the end: does this reply go out automatically, or does a human see it first?

Escalate — send to a human instead of auto-sending.

Audit trail — the complete record of everything that happened, so any decision can be reconstructed later. Ours is `logs/*.jsonl`.

Mock mode — `--mock` uses fake canned responses instead of calling the real AI. Free, instant, no key needed.

Stub — a placeholder function that returns something reasonable so the project runs before the real version is written. Most of your job is replacing stubs.

AI terms

LLM (large language model) — the AI itself. We're using DeepSeek.

API — how our code talks to the AI over the internet.

API key — the password that proves we're allowed to use it. **Never commit it.** It lives in `.env`, which git ignores.

Token — roughly a word-piece. Models charge per token, so tokens are how cost gets measured.

Temperature — how random the model's output is. `0.1` is consistent and predictable, `0.9` is creative and varied. Triage and critic run low; the drafter runs a bit warmer.

Structured output — making the model return a strict data shape instead of free text. LangChain's `with_structured_output()` does this, and it's why our agent files are so short.

Hallucination — the model confidently inventing something false. In our project, the drafter inventing a policy that doesn't exist. The critic's main job is catching this.

LangChain — the library we use to talk to the model.

Pydantic — the library that defines and validates our data shapes. `state.py` is all Pydantic.

JSONL — a file with one JSON object per line. Our log format. Easy to read line by line, and a crash mid-run doesn't corrupt what's already written.

Git, for people who haven't used it

You need about six commands. That's it. Everything else you can look up later.

The one rule

Never commit directly to `main`. You work on your own branch, you open a pull request, Enrique merges it. That's the whole system, and it's what keeps `main` working at all times.

Why this matters: if `main` breaks the night before the demo, we have nothing to show. If your *branch* breaks, only you are stuck.

First time only

```
git clone https://github.com/<enrique>/northstar-triage.git
cd northstar-triage
pip install -r requirements.txt
python main.py --mock
```

If that last command prints tickets, you're set up correctly.

Every time you sit down to work

```
git checkout main          # go to the main branch
git pull                  # get everyone else's latest work
git checkout -b yourname/what-youre-doing
```

Branch names: `nataki/triage-agent`, `julian/critic-prompt`, `lance/csv-loader`. Your name first so we can tell whose branch is whose at a glance.

Always `git pull` on main before making a new branch. Skipping this is the number one cause of ugly merge conflicts.

While you work

```
git status                # what have I changed? run this constantly
git add .                  # stage everything you changed
git commit -m "Add JSON validation to triage agent"
git push -u origin nataki/triage-agent
```

Commit often. Small commits are easier to undo than one giant one.

Write commit messages that finish the sentence "this commit will...":

- Good: `Add fallback when model returns unknown category`
- Bad: `updates`, `fix`, `asdfgh`

The `-u origin <branch>` part is only needed the first time you push a new branch. After that, plain `git push` works.

When you're done with a piece

1. Go to the repo on github.com
2. You'll see a banner: "yourname/your-branch had recent pushes" → click **Compare & pull request**
3. Write a sentence about what you did
4. Click **Create pull request**
5. Tell the group chat
6. Enrique reviews and merges

Then start fresh: `git checkout main`, `git pull`, new branch.

Things that will happen

"I have uncommitted changes and I need to switch branches."

```
git stash          # put your changes in a drawer
git checkout main
git stash pop      # take them back out
```

"I committed to main by accident." Don't panic and don't try to fix it yourself. Message Enrique. It's recoverable and it's much easier to fix before anyone else pulls.

"Git is asking me to write a merge message in a weird editor." That's vim. Type `:wq` and press Enter.

"I pulled and now there are <<<<<< markers in my file." That's a merge conflict. The markers show your version and someone else's version of the same lines. Delete the markers, keep the correct code, save, then `git add` and `git commit`. If you're unsure which version is right, ask before guessing.

"I want to throw away my changes and start over."

```
git checkout -- .      # discards uncommitted changes. Can't be undone.
```

The thing that will actually cause a problem

Never commit `.env`. It has our API key in it. Once a key is in git history it's compromised even if you delete the file afterward — the key has to be rotated and every commit that touched it lives forever in the log.

`.env` is already in `.gitignore`, so this should be impossible. But if you ever see `.env` in the output of `git status`, stop and message Enrique before committing.

Enrique's job (merging PRs)

```
git checkout main
git pull
git checkout their-branch
python main.py --mock          # does it still run?
```

If it runs, merge on github.com. If it doesn't, comment on the PR with the error and let them fix it on their branch. **Do not fix it yourself on main.**

Once a week, or before any big deadline, tag a known-good state:

```
git tag -a v0.1 -m "Working pipeline, mock mode, all agents stubbed"
git push origin v0.1
```

That gives us a snapshot we can return to if something goes badly wrong later.

The handful of things in this codebase you might not have seen yet

If you've done an intro Python course, you know everything here *except* roughly six constructs. Here they are, explained in terms of our actual code. Read this once and the whole repo opens up.

1. Pydantic models — a labeled container that checks itself

Where: `state.py`

You already know dictionaries:

```
ticket = {"ticket_id": "NS-1001", "subject": "Charged twice"}
print(ticket["subect"])      # typo -> KeyError at runtime, maybe hours later
```

A Pydantic model is the same idea, with the field names and their types locked in ahead of time:

```
class Ticket(BaseModel):
    ticket_id: str
    customer_name: str
    subject: str

ticket = Ticket(ticket_id="NS-1001", customer_name="Dana", subject="Charged twice")
print(ticket.subject)      # dot instead of brackets
print(ticket.subect)      # typo -> your editor underlines it before you run
```

`BaseModel` is the Pydantic base class. Inheriting from it is what gives you the validation.

The part that matters for us. Look at `TriageResult`:

```
Category = Literal["billing", "technical", "returns", "other"]

class TriageResult(BaseModel):
    category: Category
    confidence: float = Field(ge=0.0, le=1.0, description="...")
```

`Literal[...]` means only those exact four strings are allowed. `ge=0.0, le=1.0` means confidence must be between zero and one. If the model tries to return `"refunds"` or `1.7`, Pydantic rejects it immediately with a clear error naming the field.

That validation is **the reason our agent files are so short**. Nataki doesn't have to write "if category not in CATEGORIES" checks in `triage.py`, because the check already happened.

`Field(description=...)` is not a comment. LangChain sends those descriptions to the model as part of the request. Rewriting a description is one of the most effective ways to change how the agent behaves. That's a real tuning knob, not documentation.

2. `str, -> TriageResult, Optional[...]` — type hints

Where: everywhere, in function signatures

```
def run_triage(ticket: Ticket) -> TriageResult:
```

This reads as: "takes a `Ticket`, gives back a `TriageResult`."

Python does not enforce any of this. It's documentation that your editor can read. If you delete every type hint in the project, the code runs identically. They're there so you can tell what a function wants without reading its body.

`Optional[TriageResult]` means "a `TriageResult`, or `None`." We use it in `TicketRun` because when a ticket is first created, triage hasn't run yet, so that field starts empty.

3. `Field(default_factory=list)` — a fix for one specific Python trap

Where: `state.py`, in `TicketRun` and `CriticVerdict`

```
revision_notes: list[str] = Field(default_factory=list)
```

The trap: if you write `revision_notes: list = []`, Python creates **one** list and shares it across every object you ever make. Ticket 1's revision notes would show up in Ticket 2. This is a famous Python gotcha and it produces bugs that take hours to find.

`default_factory=list` means "make a fresh empty list for each one."

`list[str]` means "a list of strings" — Pydantic will reject a list with a number in it.

What you need to do: nothing. Just don't change it to `= []`.

4. `os.path.dirname(__file__)` — finding files reliably

Where: `tools/policy_lookup.py`, `logging_utils.py`, `llm_client.py`

```
KB_PATH = os.path.join(os.path.dirname(os.path.dirname(__file__)), "data", "policy_kb.md")
```

Ugly, but it solves a real problem. If you write `open("data/policy_kb.md")`, that path is relative to **wherever you were standing when you ran the command**, not to where the file lives. Run the script from a different folder and it breaks.

Reading it inside out:

- `__file__` — the full path of the file this line is written in
- `os.path.dirname(...)` — chop off the filename, leaving the folder
- wrapped twice — go up two folders (from `tools/` to the project root)
- `os.path.join(root, "data", "policy_kb.md")` — build the path, using the right slash for Windows or Mac

What you need to do: copy the pattern if you add a new data file. Don't hand-write paths with slashes in them.

5. `with open(...) as f:` — reading and writing files

Where: `logging_utils.py`, `tools/policy_lookup.py`, `tools/ticket_source.py`

```
with open(log_path, "a", encoding="utf-8") as f:
    f.write(json.dumps(event) + "\n")
```

Three parts:

- `open(path, mode)` — the mode is one letter. `"r"` read, `"w"` write (erases what was there), `"a"` append (adds to the end). Our log uses `"a"` so each event adds a line instead of wiping the file.
- `as f` — `f` is now the open file. Read it with `f.read()`, write with `f.write()`.
- `with` — closes the file automatically when the block ends, even if something crashes partway. Without it you can lose data that was still sitting in a buffer. Always use `with`.

`encoding="utf-8"` tells Python to expect any character, including accents and emoji. Leaving it off works on a Mac and breaks on Windows, which is a genuinely annoying bug to chase down. Just always include it.

Reading line by line:

```
with open(log_path, encoding="utf-8") as f:
    for line in f:
        event = json.loads(line)
```

`json.dumps` turns a Python dict into a string. `json.loads` turns it back. Dump to disk, load from disk.

6. `**payload` — merging one dict into another

Where: `logging_utils.py`

```
event = {
    "timestamp": ...,
    "event_type": event_type,
    **payload,          # <- unpack whatever's in payload, right here
}
```

If `payload` is `{"category": "billing", "confidence": 0.9}`, those two keys get poured into `event` alongside the fixed ones. It saves writing a loop.

It's why every agent can log completely different fields without us having to predict them in advance.

7. `with_structured_output()` — the LangChain piece that matters

Where: `llm_client.py`, used by `agents/triage.py` and `agents/critic.py`

```
model = get_model(temperature).with_structured_output(TriageResult)
result = model.invoke([("system", prompt), ("human", user_message)])
# result is a TriageResult object. Not a string. Not a dict.
```

Without this, talking to a model looks like: get a string back, hope it's JSON, strip the ````json` fences the model added anyway, `json.loads` it, pray the keys are what you expected, convert the types by hand. Every one of those steps is a place a beginner project crashes at 11pm.

`with_structured_output(TriageResult)` sends our Pydantic class to the API as a schema and hands back a real, validated `TriageResult`. The parsing step is gone entirely.

The drafter doesn't use it, because a customer reply is prose, not a data structure. It uses `call_text()` instead. That asymmetry is intentional.

What we deliberately did NOT use

`create_agent`. LangChain can build an agent loop where the *model* decides which tool to call next. We didn't, and this is worth saying out loud in the pitch rather than hiding.

Our pipeline is a fixed sequence: triage, then policy, then draft, then critic, every time. A model-driven loop could decide to skip the critic on some runs. "Every reply gets reviewed before it ships" is the core promise of our business case, and explicit orchestration is what guarantees it.

Knowing when *not* to reach for a tool is a real engineering skill. We used LangChain for structured outputs, model configuration, and tracing, where it genuinely removes work. We kept the control flow in plain Python, where it genuinely matters that the sequence can't vary.

Also not here: `async` / `await`, inheritance beyond `BaseModel`, decorators, deeply nested comprehensions.

When you're stuck

1. Run `python main.py --mock` and see if the pipeline still works
2. Add a `print()` — it's not cheating, it's how everyone debugs
3. Paste the whole error message into a chat with the group, not just the last line. The stack trace above the error usually names the file and line.

YOUR SECTION

Enrique

Orchestrator Engineer & Repo Owner

Enrique — Orchestrator Engineer & Repo Owner

You own: `orchestrator.py`, `state.py`, `main.py`, the repo, the architecture diagram.

Your job in one sentence: decide who runs when, what each agent gets handed, and where a ticket goes at the end. Plus keep `main` working so the team always has solid ground to stand on.

You have the hardest technical piece and the coordination load. Some of your tasks below are unblocking other people, so do those first even though they aren't coding.

Task 1 — Get the repo live (do this today, everyone is waiting)

1. Go to github.com, click **New repository**
2. Name: `northstar-triage`. **Public** (the assignment requires a public link). Do **not** check "Add a README" — we already have one.
3. Copy the URL it gives you.
4. In your terminal:

```
cd northstar-triage
git init
git add .
git commit -m "Initial scaffold: pipeline, agents, docs"
git branch -M main
git remote add origin https://github.com/<you>/northstar-triage.git
git push -u origin main
```

1. On github.com: **Settings** → **Collaborators** → **Add people**. Add all four teammates. They each get an email invite they have to accept.
2. Tag this known-good state so we can always come back to it:

```
git tag -a v0.1 -m "Working scaffold, mock mode, all stubs green"
git push origin v0.1
```

1. Post the repo link in the group chat along with the Day 0 instructions from `docs/START_HERE.md`.

Check yourself: open the repo URL in a private browser window. If you can see the files without logging in, it's public and the submission link will work.

Task 2 — Sort out the API key (unblocks Mitchy and Julian)

They cannot tune prompts against canned mock responses. They need real calls.

1. Confirm your DeepSeek key works:

```
cp .env.example .env
# paste your key into .env
python main.py --limit 2
```

You should see real replies, not text starting with "MOCK".

1. Decide with the group how everyone gets access. Two options: - **Shared key** (simplest): send it over DM, never in the repo, never in a screenshot. Everyone pastes it into their own local `.env`. - **Own keys**: DeepSeek is cheap and each person makes their own. Safer, slightly more setup friction.
2. Whichever you pick, say clearly in the chat: `.env` **never gets committed**. It's already in `.gitignore`, so this should be impossible, but say it anyway.

Rough cost check: 12 tickets is roughly 30 model calls. On DeepSeek that's fractions of a cent. Nobody needs to be afraid of running it. Tell them that, because people who are scared of the cost will avoid testing.

Task 3 — Run the kickoff meeting (60–90 minutes)

Put `docs/architecture.png` on screen and walk the pipeline left to right. Then get decisions on these three things. **Write the answers down** — they go in the pitch, and "we discussed it" isn't the same as "we decided."

Decision 1: what forces a human review? The current list is in `prompts/critic.md`. Legal threats, refunds over \$100, safety issues, cancellation requests, category mismatch. Does the group agree? Each line you add makes the system safer and less useful. That tradeoff *is* the project.

Decision 2: how many retries? Currently `MAX_ATTEMPTS = 2` in `orchestrator.py`. First draft, one revision, then escalate. Higher costs more and delays the ticket. Justify whatever number you land on.

Decision 3: what does the drafter get to see? Right now it gets the ticket and the policy, but **not** the triage agent's reasoning. The reasoning is for the audit trail, so it can't bias the draft. This is the "handoff design" bullet in the assignment. Make sure everyone can explain it, because a grader may ask any one of you.

Also at this meeting: schedule the ticket-labeling hour from Julian's guide.

Task 4 — Understand your own file

Open `orchestrator.py` and read `process_ticket()` top to bottom. It's about 80 lines and it's the spine of the whole project. Four sections:

1. Triage
2. Policy lookup
3. The draft/critique/retry loop
4. The approval gate

Trace one ticket by hand. Add prints and run with `--limit 1`:

```
print(f">>> triage says: {run.triage}")
print(f">>> policy is {len(run.policy_text)} chars")
print(f">>> draft attempt {run.attempts}")
print(f">>> verdict: {run.verdict}")
```

```
python main.py --mock --limit 1
```

Watching the object fill up step by step is the fastest way to actually understand this. Delete the prints when you're done.

Task 5 — The retry loop is yours to make correct

This is the trickiest logic in the project. Currently:

```
while run.attempts < MAX_ATTEMPTS:
    run.attempts += 1
    run.draft = run_drafter(..., revision_notes=run.revision_notes)
    run.verdict = run_critic(...)
    if run.verdict.approved:
        break
    run.revision_notes = run.verdict.issues
```

Verify these three behaviours in mock mode (the mock critic rejects on its second call, so this is testable for free):

1. An approved draft breaks out after 1 attempt
2. A rejected draft comes back with `attempts == 2`
3. A draft rejected twice ends up `awaiting_human`, not silently sent

Confirm #3 especially. A bug that ships an unapproved reply is the one failure that would sink the pitch.

One improvement worth making: `revision_notes` currently gets overwritten each attempt. Consider appending instead, so the drafter sees the whole history of complaints. Try both, see which produces better revisions.

Task 6 — The approval gate

Bottom of `process_ticket()`. Three outcomes: `auto_sent`, `awaiting_human`, `failed`.

Right now **any** `needs_human` flag routes to a person. The alternative the assignment hints at is tiering: auto-send low-value replies, batch-approve medium ones, individually review high-value ones.

You don't have to build tiering. But you do have to be able to say why you didn't, and "we chose the conservative version for v1 because a wrong reply costs more than a slow one" is a perfectly good answer.

Make sure the log records the reason. Every escalation writes `escalated_to_human` with a `reason` field. That's what proves the gate is working when someone asks.

Task 7 — Merging pull requests (your weekly rhythm)

When someone opens a PR:

```
git checkout main
git pull
git checkout their-branch-name
python main.py --mock          # does it still run?
```

- **Runs fine** → merge it on github.com, tell them in chat
- **Broken** → comment on the PR with the error, let them fix it on their branch. **Do not fix it yourself on main.** Fixing their code for them is faster once and slower every time after.

Merge little and often. A PR sitting open for four days is a merge conflict waiting to happen.

Tag a new known-good state after any big merge:

```
git tag -a v0.2 -m "Real API calls working end to end"
git push origin v0.2
```

Task 8 — Integration test before the demo (do NOT skip)

Two days before you present:

```
git checkout main
git pull
rm -rf logs/*.jsonl logs/approval_queue.json
python main.py          # full run, real API, all 12 tickets
python main.py --review # work the queue like a supervisor would
```

Watch for: does it finish without crashing, do the numbers look believable, does the approval queue actually contain the tickets you'd expect a human to see.

Then **rehearse the demo once, out loud, with the terminal you'll actually use**. Whoever demos should have run the exact commands at least twice before.

Your written deliverable

The architecture diagram is done ([docs/architecture.png](#)). Your other job is being able to explain the three design decisions from Task 3 when the grader asks. Practice saying each in two sentences.

Definition of done

- [] Repo public, all four collaborators added, link posted

- [] API key working, distributed, cost explained to the team
 - [] Kickoff meeting held, three decisions written down
 - [] Retry loop verified for all three behaviours
 - [] Approval gate logs a reason for every escalation
 - [] All teammate PRs merged, `main` runs clean
 - [] Full integration test passed with real API
 - [] Demo rehearsed at least twice
-

Where you'll get stuck

"Someone committed to main by accident." Recoverable, and much easier before anyone else pulls. Don't let them try to fix it alone.

"Two PRs conflict." Merge the smaller one first, then ask the second person to `git checkout main && git pull && git merge main` on their branch and resolve it there. Conflicts get resolved on branches, never on main.

"My teammate's branch breaks the pipeline." That's the system working. Main is fine, that's the whole point of branches. Comment and move on.

YOUR SECTION

Lance

Integration Engineer

Lance — Integration Engineer

You own: `tools/ticket_source.py`, `tools/policy_lookup.py`, `data/tickets.csv`

Your job in one sentence: get data into the system reliably. Tickets in, policy out, and neither one crashes when something is malformed.

Good news for a beginner: **your files never call an LLM**. No API key needed, no cost, instant feedback, fully testable. You can do your entire job in mock mode. That makes you the best-positioned person on the team to build confidence before touching anything unpredictable.

Task 1 — Read what you own (30 minutes)

```
python main.py --mock --limit 3
```

Then open both your files and read the docstrings at the top. Then try this in a Python shell to see your code in isolation:

```
python3
```

```
from tools.ticket_source import load_tickets
tickets = load_tickets(limit=2)
print(tickets[0])
print(tickets[0].subject)

from tools.policy_lookup import lookup_policy, load_policy_sections
print(load_policy_sections().keys())
print(lookup_policy("returns"))
```

Poking at functions directly in a shell, instead of running the whole program, is a skill worth having. Use it constantly.

Task 2 — Make ticket loading survive bad data

Right now a row with a missing column crashes the whole run. That's the single most likely thing to break your demo, because you're about to add a lot of rows by hand.

Prove the bug first. Open `data/tickets.csv` in a text editor, delete the `received_at` value from one row, save, and run:

```
python main.py --mock --limit 5
```

You'll get a `KeyError`. Good. Now fix it in `tools/ticket_source.py`:

```

for row_number, row in enumerate(reader, start=2): # 2 because line 1 is headers
    try:
        tickets.append(Ticket(
            ticket_id=row["ticket_id"],
            customer_name=row["customer_name"],
            subject=row["subject"],
            body=row["body"],
            received_at=row["received_at"],
        ))
    except Exception as e:
        print(f" WARNING: skipping row {row_number} in tickets.csv - {e}")
        continue

    if limit and len(tickets) >= limit:
        break

```

Run again. You should now see your warning, and the other tickets should process normally. **Then put the deleted value back** so the data is clean.

Why this matters for the pitch: a real support inbox has malformed data in it constantly. "We handle bad rows without dropping the batch" is a real engineering answer to a real question.

Task 3 — Grow the ticket set to ~30 (this is your biggest contribution)

We have 12. That's enough to run, not enough to measure. Julian needs volume to compute accuracy, and the demo is more convincing with a realistic inbox.

Open `data/tickets.csv` and add rows. Keep the same five columns. **Keep IDs sequential** (NS-1013, NS-1014...).

Aim for this mix:

Kind	How many	Why
Clear billing	5	the easy case
Clear technical	5	the easy case
Clear returns	5	the easy case
Genuinely ambiguous	4	should produce LOW confidence, and route to a human
Legal threat / very angry	2	must trigger <code>needs_human</code>
Refund over \$100	2	must trigger <code>needs_human</code>
Doesn't fit any category	2	should classify as "other"
Vague one-liners	3	"it's broken. help." — tests the hard case

Write them like real people write. Typos, missing details, run-on sentences, all lowercase. A ticket set where every message is a clean paragraph will make our accuracy look better than it is, and that's the kind of thing a grader notices.

Careful with commas. CSV uses commas as separators, so any field containing a comma must be wrapped in double quotes:

```
NS-1013,Maya Chen,Refund please,"I ordered this on the 3rd, it never came.",2026-08-04T09:00:00
```

Verify after every batch you add:

```
python -c "from tools.ticket_source import load_tickets; t=load_tickets(); print(f'{len(t)} tickets loaded')"
```

If the number is lower than what you added, a row is malformed. Your Task 2 warning will tell you which line.

Task 4 — Handle the policy lookup miss

In `tools/policy_lookup.py`, decide what happens when a category has no policy section. Two things to fix:

Normalize the input so a stray capital or space doesn't cause a miss:

```
def lookup_policy(category):
    sections = load_policy_sections()
    key = (category or "").strip().lower()
    return sections.get(key, "NO POLICY FOUND for this category. Escalate to a human agent.")
```

Test the miss path deliberately:

```
from tools.policy_lookup import lookup_policy
print(lookup_policy("nonsense"))      # should return the escalate message
print(lookup_policy(" BILLING "))     # should still find the billing policy
```

Then talk to Julian. When the policy is missing, the drafter writes from nothing, which is exactly what his critic should catch. One of you has to own that case. Agree out loud who does, and write it down.

Task 5 (stretch, worth real credit) — SQLite version

The assignment likes "one tool wired to a real data source." A CSV is defensible, but SQLite is a database, ships with Python, needs no install, and makes the claim stronger.

Create `tools/build_db.py`:

```

"""One-time script: load tickets.csv into a SQLite database."""
import csv
import os
import sqlite3

HERE = os.path.dirname(os.path.dirname(__file__))
DB_PATH = os.path.join(HERE, "data", "tickets.db")
CSV_PATH = os.path.join(HERE, "data", "tickets.csv")

conn = sqlite3.connect(DB_PATH)
conn.execute("DROP TABLE IF EXISTS tickets")
conn.execute("""
    CREATE TABLE tickets (
        ticket_id    TEXT PRIMARY KEY,
        customer_name TEXT,
        subject      TEXT,
        body         TEXT,
        received_at   TEXT
    )
""")

with open(CSV_PATH, newline="", encoding="utf-8") as f:
    rows = [
        (r["ticket_id"], r["customer_name"], r["subject"], r["body"], r["received_at"])
        for r in csv.DictReader(f)
    ]

conn.executemany("INSERT INTO tickets VALUES (?, ?, ?, ?, ?)", rows)
conn.commit()
print(f"Loaded {len(rows)} tickets into {DB_PATH}")

```

Run it once:

```
python tools/build_db.py
```

Then add a loader alongside the CSV one in `ticket_source.py`:

```

def load_tickets_from_db(limit=None):
    """Same as load_tickets, but reads from SQLite."""
    import sqlite3
    db_path = os.path.join(os.path.dirname(os.path.dirname(__file__)), "data", "tickets.db")
    conn = sqlite3.connect(db_path)
    conn.row_factory = sqlite3.Row
    query = "SELECT * FROM tickets ORDER BY received_at"
    if limit:
        query += f" LIMIT {int(limit)}"
    rows = conn.execute(query).fetchall()
    conn.close()
    return [Ticket(**dict(row)) for row in rows]

```

Keep both. Same interface, two sources, which is exactly the point you make in the pitch: swapping the CSV for Zendesk's API later touches only this file.

Add `data/tickets.db` to `.gitignore` — it's generated, not source.

Definition of done

- [] A malformed CSV row prints a warning and gets skipped, run continues
 - [] ~30 tickets covering every category plus the edge cases in Task 3
 - [] `lookup_policy` handles a missing category and messy capitalization
 - [] You and Julian have agreed who owns the missing-policy case
 - [] (Stretch) SQLite loader working, same interface as the CSV one
 - [] Your branch merged into main
-

Your line in the pitch

"The pipeline reads from a SQLite database through a single interface. Swapping in a real helpdesk API touches one file and nothing else. Malformed records are logged and skipped rather than dropping the whole batch."

Where you'll get stuck

KeyError: 'received_at' — a CSV row is missing a column, or the header row got edited. Your Task 2 fix turns this into a warning.

A row with a comma in it splits into extra columns — wrap that field in double quotes.

FileNotFoundError — you're running from the wrong folder. Always run from the project root, the folder with `main.py` in it.

Excel mangles the CSV — it likes to reformat dates and strip quotes. Use a plain text editor (VS Code, Notepad++, TextEdit in plain mode) instead.

YOUR SECTION

Mitchy

Prompt Engineer

Mitchy — Prompt Engineer

You own: `prompts/triage.md`, `prompts/drafter.md`, `agents/drafter.py`, `data/policy_kb.md`, and the Before/After one-pager.

Your job in one sentence: the agents do what you tell them, so you decide what they're told and what they're allowed to know.

You need real API calls to do your job well. Mock responses are canned, so they can't tell you whether a prompt is good. Get the API key from Enrique early.

Task 1 — See what the drafter currently produces (30 minutes)

```
cp .env.example .env
# paste the key Enrique gives you
python main.py --limit 3
```

Then read the actual drafts in the log:

```
python -c "
import json, glob
path = sorted(glob.glob('logs/*.jsonl'))[-1]
for line in open(path):
    e = json.loads(line)
    if e['event_type'] == 'draft_created':
        print('=' * 60)
        print(f'{e["ticket_id"]} attempt {e["attempt"]}\n')
        print(e['draft'])
"
```

Read them like a customer would. Too formal? Too vague? Does it promise something the policy didn't authorize?

Write down three specific complaints. Those are your first three fixes.

Task 2 — Understand your two tuning surfaces

This trips people up, so read it carefully. Your prompts live in **two** places.

1. `prompts/*.md` — persona, rules, tone, what to do and not do. Normal prompt writing.
2. `Field(description=...)` in `state.py` — LangChain sends these to the model as part of the schema. They describe individual output fields.

```
confidence: float = Field(
    ge=0.0, le=1.0,
    description="How certain you are, 0.0 to 1.0. Be honest - if the ticket "
    "could reasonably be two categories, say 0.6, not 0.95."
)
```

That description is doing prompt work. Rewriting it changes agent behaviour.

state.py is Enrique's file. Don't edit it directly, or you'll create a merge conflict. Send him the exact replacement wording and let him commit it. Annoying, but one file with one owner is what keeps this project from breaking.

Rule of thumb: how the agent *behaves* goes in the prompt file. What a *field means* goes in the Field description.

Task 3 — Expand the policy knowledge base

Open `data/policy_kb.md`. The drafter can only state policy that appears here. If it's thin, the drafter either says "I'll pass you to a specialist" constantly or invents things, and Julian's critic rejects it.

Structural rule you cannot break: the `## header` names must exactly match the categories in `state.py` — `billing`, `technical`, `returns`, `other`. Lance's lookup matches on those headers. Rename one and the policy silently goes missing.

Add specifics for each category. Real numbers, real timeframes. Every ticket Lance writes should have a policy that answers it.

Verify after editing:

```
from tools.policy_lookup import load_policy_sections
s = load_policy_sections()
print(list(s.keys()))          # must be exactly the four category names
print(len(s['billing']), 'chars of billing policy')
```

Task 4 — Tune the drafter prompt, one change at a time

Open `prompts/drafter.md`. The workflow:

1. Change **one** thing
2. `python main.py --limit 3`
3. Read the drafts
4. Better or worse?
5. Repeat

One change at a time. Change three things and you won't know which one helped. This is slow and it is the actual job.

Things worth trying:

- **Add examples.** Two or three "here's a ticket, here's a good reply" pairs usually beat any amount of rule-writing. This is the single highest-leverage change available to you.
- **Tighten rule 2** (never invent a number). It's the rule that breaks most often and the one Julian's critic will reject you for.
- **Test length.** Try "three to five sentences" versus "four to six" and see what the critic says.

- **Test the escalation path.** Feed it a ticket with no matching policy. Does it correctly say it's passing to a specialist, or does it make something up? Making something up is the worst failure mode we have.

Keep notes on what you tried. Some of it goes in the pitch.

Task 5 — Make the retry path actually work

This is the piece most likely to be quietly broken, and it's the "retry logic" bullet in the assignment rubric.

When Julian's critic rejects a draft, `run_drafter` gets called again with `revision_notes` filled in. **The common failure is the model politely ignoring the notes and returning a near-identical draft.**

Test it directly:

```
from state import Ticket, TriageResult
from agents.drafter import run_drafter

ticket = Ticket(ticket_id="T1", customer_name="Dana", subject="Refund",
                 body="I was charged twice for the same order.",
                 received_at="2026-08-04T09:00:00")
triage = TriageResult(category="billing", priority="normal",
                      confidence=0.9, reasoning="test")
policy = "Duplicate charges are refunded in full within 60 days. Refunds take 3-5 business days."

first = run_drafter(ticket, triage, policy)
print("FIRST DRAFT:\n", first)

second = run_drafter(ticket, triage, policy,
                     revision_notes=["The reply never tells the customer how long the refund
takes."])
print("\nREVISED DRAFT:\n", second)
```

The second draft must visibly fix the named problem. If it doesn't, strengthen the revision instructions in `prompts/drafter.md`. Something like: *"You are revising a rejected draft. Address every listed problem explicitly. Do not rewrite parts that were not criticized."*

Task 6 — Your written deliverable: the Before/After one-pager

Current process, from the assignment:

- 6 human agents
- reads every ticket, tags, looks up policy, drafts a reply
- supervisor reviews every reply
- **18 hour average resolution time**
- can't scale in seasonal spikes without temp hires

After, from our actual run. **Use real numbers from the logs, not guesses.** Get these from Nataki's run summary:

- automation rate (what % went out without a human)

- average attempts per ticket
- cost per ticket
- wall-clock time per ticket

For staff hours, state your assumptions in the document. Something like: "at an estimated 12 minutes of agent time per ticket and 3 minutes of supervisor review, 100 tickets/day is roughly 25 staff-hours. At a 70% automation rate, supervisor review drops to the flagged 30%."

Include the risk column. A one-pager that only shows savings reads as a sales pitch. What we're trading: a wrong reply can now reach a customer without a human seeing it. Say what mitigates that (the critic, the flag rules, the audit log) and say plainly what it doesn't fully solve.

Definition of done

- [] Policy KB covers every ticket category with real specifics
- [] Section headers exactly match the four category names
- [] Drafter prompt has worked examples in it
- [] Retry path verified: a revised draft visibly fixes the named problem
- [] Drafter correctly escalates when no policy matches instead of inventing
- [] Before/After one-pager written, with real numbers and a risk section
- [] Your branch merged into main

Your line in the pitch

"The drafter can only state policy that was retrieved for it. When no policy matches, it escalates instead of guessing. We tested that path deliberately rather than assuming it."

Where you'll get stuck

"Policy comes back as NO POLICY FOUND." A `##` header in `policy_kb.md` doesn't match a category name exactly. Check spelling and capitalization.

"My prompt change did nothing." Two likely causes: the temperature is high enough that you're seeing noise (run the same ticket 3 times to check), or the behaviour you're trying to change lives in the Field description in `state.py`, not the prompt file.

"Every draft gets rejected by the critic." Talk to Julian before changing anything. It may be his rules that are too strict, not your prompt. This is a conversation, not a solo debugging session.

"It's slow." Real API calls take a few seconds each. Use `--limit 3` while you're iterating.

YOUR SECTION

Julian

QA / Critic Engineer

Julian — QA / Critic Engineer

You own: `agents/critic.py`, `prompts/critic.md`, the labeled evaluation set, and the 5-minute pitch deck.

Your job in one sentence: you are the supervisor's second look. Everything the team claims about quality has to be something your critic actually enforces.

You also own the only numbers in this project. Right now nobody can answer "is it any good?" After Task 3, you can. That number goes on the pitch slide and it's the most valuable thing anyone on this team produces.

Task 1 — Understand the two-boolean design (read this twice)

The critic returns two separate flags:

```
approved: bool      # is the WRITING good enough to send?
needs_human: bool   # should a PERSON see it regardless?
```

A reply can be beautifully written and still need a human. Customer mentions a lawyer, the draft is polite and accurate, and it still must not auto-send.

Collapsing these into one flag is the obvious first instinct and it's wrong. This distinction is basically the whole answer to the assignment's "which human checkpoint survives" question. **Make sure you can explain it in two sentences**, because it's the thing a grader is most likely to probe.

Task 2 — Set the rules

Open `prompts/critic.md`. Two lists.

Rejection rules (send it back to Mitchy's drafter to fix):

1. States a policy not in the retrieved policy text
2. Contains a number, date, or timeframe it was never given
3. Promises something the policy doesn't authorize
4. Doesn't answer what the customer asked
5. Tone would upset an already-frustrated customer

Escalation rules (`needs_human = true`, even when well written):

- legal threat, safety issue, refund over \$100, cancellation request, category mismatch

Bring both lists to the group meeting. These are business rules, not technical ones, and the whole team has to agree. Every line you add makes the system safer and less useful. That tradeoff is the project.

Write specific issues. The drafter only sees your `issues` list, not your reasoning:

- Useless: "the tone is off"
- Useful: "the closing line is dismissive; the customer already said they tried restarting"

Task 3 — Build the labeled set (schedule this early)

Nobody can measure anything until this exists.

Run a one-hour group session. Five people, Lance's ~30 tickets, everyone labels 6. You own the file.

Create `data/labeled_tickets.csv`:

```
ticket_id,expected_category,expected_needs_human,notes
NS-1001,billing,FALSE,clear duplicate charge
NS-1006,billing,TRUE,mentions contacting a lawyer
NS-1008,returns,TRUE,refund over $100
NS-1010,other,FALSE,general pricing question
```

Rules for the session:

- Label what the answer *should* be, without looking at what the system said. Looking first will bias you and the number becomes meaningless.
- Disagreements are the valuable part. If two of you disagree on a ticket, that ticket is genuinely ambiguous, which means the agent should return **low confidence** on it. Note those.
- Don't skip the hard ones. A labeled set of only easy tickets produces a flattering, useless accuracy number.

Task 4 — Write the scoring script

Create `scripts/evaluate.py`. This is your headline deliverable.

```

"""
Measure triage accuracy against our hand-labeled set.

Usage:
python scripts/evaluate.py --mock      # sanity check, meaningless numbers
python scripts/evaluate.py            # the real number
"""

import argparse
import csv
import os
import sys

sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

parser = argparse.ArgumentParser()
parser.add_argument("--mock", action="store_true")
args = parser.parse_args()
if args.mock:
    os.environ["NORTHSTAR MOCK"] = "1"

from agents.triage import run_triage
from tools.ticket_source import load_tickets

HERE = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
LABELS = os.path.join(HERE, "data", "labeled_tickets.csv")

with open(LABELS, newline="", encoding="utf-8") as f:
    expected = {r["ticket_id"]: r["expected_category"] for r in csv.DictReader(f)}

tickets = [t for t in load_tickets() if t.ticket_id in expected]

correct = 0
wrong = []
low_confidence = 0

for ticket in tickets:
    result = run_triage(ticket)
    want = expected[ticket.ticket_id]
    if result.category == want:
        correct += 1
    else:
        wrong.append((ticket.ticket_id, want, result.category, result.confidence))
        if result.confidence < 0.70:
            low_confidence += 1

total = len(tickets)
print(f"\nTriage accuracy: {correct}/{total} = {correct / total * 100:.1f}%")
print(f"Low confidence (routed to human): {low_confidence}/{total}")

if wrong:
    print("\nMisclassified:")
    for tid, want, got, conf in wrong:
        print(f"  {tid}: expected {want}, got {got} (confidence {conf:.2f})")
    print("\nCheck whether the wrong ones had LOW confidence. If they did, the")
    print("system caught its own mistake and routed to a human. That's a good")
    print("result and worth saying out loud in the pitch.")

```

Run it:

```
python scripts/evaluate.py --mock    # confirms the script works
python scripts/evaluate.py          # the real number
```

The insight to look for: if the misclassified tickets mostly had low confidence, the system knew it was unsure and escalated. That's a far more interesting finding than raw accuracy, and it's the kind of thing that makes a pitch land.

Task 5 — Prove your critic actually catches things

A critic that approves everything is worse than no critic, because it creates false confidence. Test it deliberately.

Deliberately break the drafter, temporarily:

1. Open `prompts/drafter.md`
2. Add: `Always tell the customer they have 90 days to return any item.`
3. That contradicts the 14-day policy, so your critic must reject it
4. `python main.py --limit 5`
5. Check the log for rejections:

```
python -c "
import json, glob
for line in open(sorted(glob.glob('logs/*.jsonl'))[-1]):
    e = json.loads(line)
    if e['event_type'] == 'critic_verdict' and not e['approved']:
        print(e['ticket_id'], '->', e['issues'])
"
```

1. **Put the drafter prompt back.** Tell Mitchy what you did first.

If the critic didn't catch it, your rules are too loose. This test is a real result: "we validated the critic by injecting a known policy violation and confirming it was caught in N of N cases" is a strong line in the deck.

Task 6 — Tune the confidence floor and measure the tradeoff

In `agents/critic.py`:

```
CONFIDENCE_FLOOR = 0.70
```

Run the pipeline at 0.60, 0.70, and 0.80 and record the automation rate each time. You'll get a table like:

Floor	Auto-sent	To human
0.60	78%	22%
0.70	65%	35%
0.80	41%	59%

That table is your best pitch slide. It shows the business the dial they actually control and lets them pick their own risk tolerance. Being able to say "you tell us the number and we'll set it" is a much stronger position than defending a value you picked yourself.

Task 7 — Your written deliverable: the 5-minute pitch

The assignment asks four questions. Answer each directly.

What are the costs? Cost per ticket from Nataki's logs, times volume, plus whatever setup time you estimate.

What is the turnaround? From 18 hours to whatever the auto-send path measures, and be honest that flagged tickets still wait for a human.

What are the risks? Be specific rather than reassuring. A wrong reply can now reach a customer without a person seeing it. The critic reduces that, and your Task 5 test gives you a number for how well. The floor table shows the dial. Say what's still unmitigated.

Do we need anyone to manage this? Yes, and say so. Someone works the approval queue. Someone updates the policy KB when policy changes. Roughly one person part-time instead of six full-time. **Do not claim it runs itself.** Overclaiming is how pitches lose credibility in the Q&A.

Rough structure, 5 minutes is short:

1. The problem, in numbers (30s)
2. What we built, using the architecture diagram (60s)
3. Live demo or screenshots (90s)
4. Results: accuracy, automation rate, the floor table (60s)
5. Risks and what we keep from the old process (60s)
6. What it costs and who runs it (30s)

Definition of done

- [] Rejection and escalation rules agreed by the group
- [] `data/labeled_tickets.csv` with ~30 labeled tickets
- [] `scripts/evaluate.py` working, real accuracy number recorded
- [] Critic validated by injecting a known policy violation
- [] Confidence floor tradeoff table at three settings

- [] Pitch deck built and rehearsed
 - [] Your branch merged into main
-

Your line in the pitch

"We validated the critic by injecting a known policy violation and confirming it was caught. We also measured what happens at three confidence thresholds, so the business can pick its own risk tolerance rather than accepting ours."

Where you'll get stuck

"The critic approves everything." Your rules are too vague. Replace "check the reply is accurate" with the numbered specific rules. Models follow concrete instructions far better than general ones.

"The critic rejects everything and every ticket hits max retries." Too strict, or Mitchy's policy KB is too thin for the drafter to work with. Talk to her before changing your rules. Usually it's the policy, not the critic.

"Accuracy is only 60%." That is a finding, not a failure. Report it honestly, look at *which* tickets failed, and check whether they had low confidence. A system that knows when it's unsure is worth more than one that's confidently wrong. Graders reward honest measurement over inflated numbers.

"scripts/evaluate.py can't find my modules." The `sys.path.insert` line at the top handles that. Run it from the project root.

YOUR SECTION

Nataki

Logging & Observability Engineer

Nataki — Logging & Observability Engineer

You own: `agents/triage.py`, `logging_utils.py`, `approval_queue.py`, and the "What We Keep From the Old Process" writeup.

Your job in one sentence: make everything the agents do visible, and build the place where a human actually reviews what got flagged.

Two things worth knowing up front. The approval queue you build is **the thing we demo live** — it's the most visible piece in the whole presentation. And the audit trail is the core argument of the project: the supervisor stops reading every reply, but nothing becomes invisible. That's your writeup.

Task 1 — See what's already being logged (30 minutes)

```
python main.py --mock
```

Then look at what came out:

```
ls logs/
python -c "
import json, glob
path = sorted(glob.glob('logs/*.jsonl'))[-1]
for line in open(path):
    e = json.loads(line)
    print(f"\{{e['event_type']:22s} \{{e['ticket_id']}}\}")
"
```

You'll see the full life of each ticket: `triage_complete`, `policy_retrieved`, `draft_created`, `critic_verdict`, then `auto_sent` or `escalated_to_human`.

JSONL means one JSON object per line. Not one big JSON file. That matters because you can read it line by line, load it into a spreadsheet, or `grep` it, and a crash mid-run doesn't corrupt what was already written.

Task 2 — Improve the triage agent

Open `agents/triage.py`. It's short, and that's on purpose: the validation lives in the `TriageResult` schema in `state.py`, so you don't hand-write category checks.

The one thing you control here is what the model sees:

```
user_message = f"Subject: {ticket.subject}\n\nBody: {ticket.body}"
```

Experiment, don't guess. Once Julian's labeled set exists, run `python scripts/evaluate.py` before and after each change and see what actually moves the number. Things to try:

- Subject only, versus subject plus body

- Adding the customer name (does it help? probably not — test it)
- Truncating very long bodies to the first 500 characters

The other tuning surface is `prompts/triage.md`, which Mitchy owns. Coordinate with her rather than both editing it.

Task 3 — Add real metrics to the run summary

Open `logging_utils.py`. It's four plain functions, no classes:

- `start_run()` makes a new log file and hands back its path
- `log_event(log_path, ...)` writes one line to it
- `read_events(log_path)` reads the lines back
- `summarize(log_path)` turns them into numbers

`summarize()` already computes automation rate and average attempts. Your job is the metric it's missing: **elapsed wall-clock time**.

In `main.py`, log one extra event at the start of the run:

```
log_event(log_path, "run_started", "-", {})
```

Then in `summarize()`, take the timestamp of the first event and the last one, and subtract them:

```
from datetime import datetime

if events:
    started = datetime.fromisoformat(events[0]["timestamp"])
    ended = datetime.fromisoformat(events[-1]["timestamp"])
    elapsed_seconds = (ended - started).total_seconds()
else:
    elapsed_seconds = 0
```

Add `elapsed_seconds` and `seconds_per_ticket` to the dict that `summarize()` returns.

Turnaround time is one of the four questions the pitch has to answer, so we need a real number for it.

Test your changes without running the whole pipeline:

```
import logging_utils as L

path = L.start_run()
L.log_event(path, "triage_complete", "T1", {"category": "billing"})
L.log_event(path, "draft_created", "T1", {"attempt": 1, "draft": "hi"})
L.log_event(path, "auto_sent", "T1", {"draft": "hi"})
print(L.summarize(path))
```

Task 4 — Build the approval queue CLI (this is what we demo)

Open `approval_queue.py`, find `review_queue()`. Right now it prints the queue and stops. Make it interactive, so a "supervisor" can actually work through it.

```
def review_queue():
    """Interactive review loop. This is the human checkpoint."""
    queue = load_queue()
    if not queue:
        print("Approval queue is empty. Nothing waiting on a human.")
        return

    print(f"\n{len(queue)} replies waiting for human review.\n")
    decisions = []

    for i, item in enumerate(queue, start=1):
        ticket = item["ticket"]
        verdict = item.get("verdict") or {}

        print("=" * 68)
        print(f"[{i}/{len(queue)}] Ticket {ticket['ticket_id']}")
        print(f"From:      {ticket['customer_name']}")
        print(f"Subject:    {ticket['subject']}")
        print(f"\nCustomer wrote:\n {ticket['body']}")
        print(f"\nFLAGGED BECAUSE: {verdict.get('reasoning', 'unknown')}")
        if verdict.get("issues"):
            print("Open issues:")
            for issue in verdict["issues"]:
                print(f"  - {issue}")
        print(f"\nDRAFTED REPLY (attempt {item.get('attempts', '?')})")
        print("  " + item.get("draft", "(none)").replace("\n", "\n  "))
        print()

        choice = ""
        while choice not in ("a", "r", "s"):
            choice = input(" [a]pprove [r]eject [s]kip > ").strip().lower()

        decisions.append({
            "ticket_id": ticket["ticket_id"],
            "decision": {"a": "approved", "r": "rejected", "s": "skipped"}[choice],
        })
        print()

    approved = sum(1 for d in decisions if d["decision"] == "approved")
    print("=" * 68)
    print(f"Reviewed {len(decisions)}: {approved} approved, "
          f"{sum(1 for d in decisions if d['decision'] == 'rejected')} rejected, "
          f"{sum(1 for d in decisions if d['decision'] == 'skipped')} skipped")

    # TODO: write these decisions back into the log so the audit trail is
    # closed-loop. That's the strongest version of this feature: we can show
    # not just what the agent proposed, but what the human decided about it.
```

Test it:

```
python main.py --mock
python main.py --review
```

The detail that makes the demo land: showing *why* each one was flagged, right above the draft. That's the difference between "here's a list" and "here's a supervisor's actual working screen." Spend your polish time there.

Task 5 — Turn on LangSmith tracing (30 minutes, high payoff)

This gives you a professional observability dashboard for free, with zero code changes.

1. Sign up at smith.langchain.com
2. Create an API key
3. Add to your `.env`:

```
LANGSMITH_TRACING=true
LANGSMITH_API_KEY=your_key_here
LANGSMITH_PROJECT=northstar-triage
```

1. `python main.py --limit 5`
2. Open the LangSmith dashboard

You'll see every model call: the exact prompt sent, the response, how long it took, how many tokens, and what it cost. **Take screenshots.** They go in the deck and in your writeup.

Get the cost-per-ticket number from here and give it to Mitchy for the Before/After one-pager and Julian for the pitch. That number is one of the four questions the assignment requires an answer to.

Worth saying explicitly in your writeup: we have two complementary layers. Our own JSONL log is the permanent audit record we control and could hand to a compliance reviewer. LangSmith is the live debugging and cost view. Different jobs.

Task 6 — Your written deliverable: "What We Keep From the Old Process"

This is arguably the most important written piece, because it's the assignment's central question. The framing it gives you is a progression:

Full Approval → Exception-Only → Audit Trail

Structure your argument like this:

What the old process had. A supervisor read every reply before it shipped. 100% human review. Slow, but a customer never received something no person had seen.

What we automated away. Reading and tagging every ticket. Hand-searching the policy doc. Writing the first draft. Reviewing the routine replies where the critic found nothing and no flag rule fired.

What we kept, and why. A human still reviews every flagged reply. Not because a person is better at grammar, but because the flag conditions mark the cases where being wrong is expensive and hard to undo: legal exposure, large refunds, safety, an already-angry customer. Those are irreversible in a way that a slightly clumsy sentence isn't.

What replaced the rest. Every prompt, draft, verdict, and retry is logged. The supervisor no longer reads every reply *in advance*, but any reply can be reconstructed completely *afterward*. That's a different kind of oversight, not an absence of it.

Be honest about what we gave up. A wrong reply can now reach a customer without a human seeing it first. The critic reduces that risk and we measured how much (get Julian's number). It doesn't eliminate it. A writeup that admits this is more credible than one that doesn't, and a grader will notice which one you wrote.

Definition of done

- [] Run summary produces automation rate, avg attempts, and elapsed time
 - [] Approval queue CLI is interactive and shows why each item was flagged
 - [] LangSmith on, screenshots captured, cost-per-ticket shared with the team
 - [] Triage tested against Julian's labeled set
 - [] "What We Keep" writeup done, including what we gave up
 - [] Your branch merged into main
-

Your line in the pitch

"Every prompt, draft, verdict, and retry is recorded. The supervisor no longer reads every reply in advance, but any reply can be reconstructed completely afterward. We traded pre-approval for a full audit trail, deliberately."

Where you'll get stuck

"The log file is empty." `log_event` only gets called when `main.py` runs the pipeline. `--review` reads the queue, it doesn't create log events.

"`input()` doesn't work." It won't run inside a Jupyter notebook or some IDE consoles. Run it from a real terminal.

"`KeyError` when reading a log entry." Different event types carry different fields. A `triage_complete` event has no `draft` key. Use `event.get("draft")` instead of `event["draft"]`.

"LangSmith shows nothing." Tracing only fires on real API calls. Mock mode never touches the network, so there's nothing to trace.